Ocean Networks Canada's ERDDAP
Easier access to Ocean Network Canada's data

Brought to you by Ocean Networks Canada CANARIE

ERDDAP > tabledap > Documentation

Using tabledap to Request Data and Graphs from Tabular Datasets

tabledap lets you request a data subset, a graph, or a map from a tabular dataset (for example, buoy data), via a specially formed URL. tabledap uses the OPeNDAP Data Access Protocol (DAP) and its selection constraints.

The URL specifies what you want: the dataset, a description of the graph or the subset of the data, and the file type for the response.

- (easy) You can get data by using the dataset's **Data Access Form** or **Subset** form. They make the URL for you.
- (easy) You can make a graph or map by using the dataset's **Make A Graph** form. It makes the URL for you.
- (not hard) You can bypass the forms and get the data or make a graph or map by generating the URL by hand or with a computer program or script.

Tabledap request URLs must be in the form
`https://coastwatch.pfeg.noaa.gov/erddap/tabledap/datasetID.fileType{?query}`
For example,
`https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.htmlTable?longitude,latitude,time,station,wmo_platform_code,T_25&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z`
Thus, the query is often a comma-separated list of desired variable names, followed by a collection of constraints (e.g., `variable<value`), each preceded by '&' (which is interpreted as "AND").

Details:

- Requests must not have any internal spaces.
- Requests are case sensitive.
- {} is notation to denote an optional part of the request.

- **datasetID** identifies the name that ERDDAP assigned to the source website and dataset (for example, `pmelTaoDySst`). You can see a list of datasetID options available via tabledap.

- **fileType** specifies the type of table data file that you want to download (for example, `.htmlTable`). The actual extension of the resulting file may be slightly different than the fileType (for example, `.htmlTable` returns an .html file).

  Which fileType should I use?
  It's entirely up to you. In general, pick the fileType which meets your needs and is easiest to use. You will use different fileTypes in different situations, e.g., viewing a graph in a browser (.png) vs. viewing the data in a browser (.htmlTable) vs. downloading a data file (e.g., .nc or .csv) vs. working with some software tool (e.g., .nc or .odv). If you are going to download lots of large files, you might also want to give some weight to fileTypes that are more compact and so can be downloaded faster.

  The fileType options for downloading tabular data are:

Data fileTypes	Description	Info	Example
.asc	View OPeNDAP-style ISO-8859-1 comma-separated text.	info	example
.croissant	Download the dataset croissant schema.	info	example
.csv	Download a ISO-8859-1 comma-separated text table (line 1: names; line 2: units; ISO 8601 times).	info	example
.csv0	Download a ISO-8859-1 .csv file without column names or units. Times are ISO 8601 strings.	info	example
.csvp	Download a ISO-8859-1 .csv file with line 1: name (units). Times are ISO 8601 strings.	info	example
.das	View the dataset's metadata via an ISO-8859-1 OPeNDAP Dataset Attribute Structure (DAS).	info	example
.dataTable	A JSON file formatted for use with the Google Visualization client library (Google Charts).	info	example
.dds	View the dataset's structure via an ISO-8859-1 OPeNDAP Dataset Descriptor Structure (DDS).	info	example
.dods	OPeNDAP clients use this to download the data in the DODS binary format.	info	example
.esriCsv	Download a ISO_8859_1 .csv file for ESRI's ArcGIS 9.x and below (separate date and time columns).	info	example
.fgdc	View the dataset's UTF-8 FGDC .xml metadata.	info	example

.geoJson	Download longitude,latitude,otherColumns data as a UTF-8 GeoJSON .json file.	info	example
.htmlTable	View a UTF-8 .html web page with the data in a table. Times are ISO 8601 strings.	info	example
.iso19115	View the dataset's ISO 19115/19139 UTF-8 .xml metadata using the version default for this server.	info	example
.iso19115_2	View the dataset's ISO 19115-2/19139 UTF-8 .xml metadata.	info	example
.iso19115_3_2016	View the dataset's ISO 19115-3:2016 UTF-8 .xml metadata.	info	example
.iso19139_2007	View the dataset's ISO 19139:2007 UTF-8 .xml metadata.	info	example
.itx	Download an ISO-8859-1 Igor Text File. Each response column becomes a wave.	info	example
.json	View a table-like UTF-8 JSON file (missing value = 'null'; times are ISO 8601 strings).	info	example
.jsonlCSV	View a UTF-8 JSON Lines CSV file without column names (mv = 'null'; times are ISO 8601 strings).	info	example
.jsonlCSV1	View a UTF-8 JSON Lines CSV file with column names on line 1 (mv = 'null'; times are ISO 8601 strings).	info	example
.jsonlKVP	View a UTF-8 JSON Lines file with Key:Value pairs (missing value = 'null'; times are ISO 8601 strings).	info	example
.mat	Download a MATLAB binary file.	info	example
.nc	Download a flat, table-like, NetCDF-3 binary file with COARDS/CF/ACDD metadata.	info	example
.ncCF	Download a NetCDF-3 CF Discrete Sampling Geometries file (Contiguous Ragged Array).	info	example
.ncCFHeader	View the UTF-8 header (the metadata) for the .ncCF file.	info	example
.ncCFMA	Download a NetCDF-3 CF Discrete Sampling Geometries file (Multidimensional Array).	info	example
.ncCFMAHeader	View the UTF-8 header (the metadata) for the .ncCFMA file.	info	example
.ncHeader	View the UTF-8 header (the metadata) for the NetCDF-3 .nc file.	info	example
.nccsv	Download a NetCDF-3-like 7-bit ASCII NCCSV .csv file with COARDS/CF/ACDD metadata.	info	example
.nccsvMetadata	View the dataset's metadata as the top half of a 7-bit ASCII NCCSV .csv file.	info	example
.ncoJson	Download a UTF-8 NCO lvl=2 JSON file with COARDS/CF/ACDD metadata.	info	example
.odvTxt	Download longitude,latitude,time,otherColumns as an ISO-8859-1 ODV Generic Spreadsheet File (.txt).	info	example
.parquet	Download as a parquet file. Metadata contains column names ("column_names") and units ("column_units").	info	example
.parquetWMeta	Download as a parquet file with detailed metadata.	info	example
.tsv	Download a ISO-8859-1 tab-separated text table (line 1: names; line 2: units; ISO 8601 times).	info	example
.tsv0	Download a ISO-8859-1 .tsv file without column names or units. Times are ISO 8601 strings.	info	example
.tsvp	Download a ISO-8859-1 .tsv file with line 1: name (units). Times are ISO 8601 strings.	info	example
.wav	Download a .wav audio file. All columns must be numeric and of the same type.	info	example
.xhtml	View a UTF-8 XHTML (XML) file with the data in a table. Times are ISO 8601 strings.	info	example

For example, here is a complete request URL to download data formatted as an HTML table:
`https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.htmlTable?longitude,latitude,time,station,wmo_platform_code,T_25&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z`

ArcGIS .esriCsv - ArcGIS is a family of Geographical Information Systems (GIS) products from ESRI: ArcView, ArcEditor, and ArcInfo. To get data from ERDDAP into your ArcGIS program (version 9.x and below):

1. In ERDDAP, save some data (which must include longitude and latitude) in an .esriCsv file (which will have the extension .csv) in the directory where you usually store ArcGIS data files. In the file:
   - Column names have been changed to be 10 characters or less (sometimes with A, B, C, ... at the end to make them unique).
   - `longitude` is renamed `X`. `latitude` is renamed `Y` to make them the default coordinate fields.
   - Missing numeric values are written as -9999.
   - Double quotes in strings are double quoted.
   - Timestamp columns are separated into date (ISO 8601) and time (am/pm) columns.
2. In ArcMap, navigate to `Tools : Add XY Data` and select the file.
3. Click on `Edit` to open the Spatial Reference Properties dialog box.
4. Click on `Select` to select a coordinate system.
5. Open the `World` folder and select `WGS_1984.prj`.
6. Click a `Add, Apply` and `OK` on the Spatial Reference Properties dialog box.

7. Click `OK` on the Add XY Data dialog box.
8. Optional: save the data as a shapefile or into a geodatabase by right clicking on the data set and choosing `Data : Export Data` ...

The points will be drawn as an Event theme in ArcMap. (These instructions are a modified version of ESRI's instructions.)

Ferret is a free program for visualizing and analyzing large and complex **gridded** datasets. Because tabledap's **tabular** datasets are very different from gridded datasets, it is necessary to use Ferret in a very specific way to avoid serious problems and misunderstandings:

1. Use the ERDDAP web pages or a program like curl to download a subset of a dataset in a plain .nc file.
2. Open that local file (which has the data in a gridded format) in Ferret. The data isn't in an ideal format for Ferret, but you should be able to get it to work.

WARNING: Ferret won't like profile data (and perhaps other data), because the time variable will have the same time value for all of the rows of data for a given profile. In fact, Ferret will add some number of seconds to the time values to make them all unique! So treat these time values accordingly.

IDL - IDL is a commercial scientific data visualization program. To get data from ERDDAP into IDL, first use ERDDAP to select a subset of data and download a .nc file. Then, use these instructions to import the data from the .nc file into IDL.

JSON .json files are widely used to transfer data to JavaScript scripts running on web pages. All .json responses from ERDDAP (metadata, gridded data, and tabular/in-situ data) use the same basic format: a database-like table. Since data from EDDTable datasets is already a table (with a column for each requested variable), ERDDAP can easily store the data in a .json file. For example,

```
{
  "table": {
    "columnNames": ["longitude", "latitude", "time", "bottle_posn", "temperature1"],
    "columnTypes": ["float", "float", "String", "byte", "float"],
    "columnUnits": ["degrees_east", "degrees_north", "UTC", null, "degree_C"],
    "rows": [
      [-124.82, 42.95, "2002-08-17T00:49:00Z", 1, 8.086],
      [-124.82, 42.95, "2002-08-17T00:49:00Z", 2, 8.585],
      [-124.82, 42.95, "2002-08-17T00:49:00Z", 3, 8.776],
      ...
      [-124.1, 44.65, "2002-08-19T20:18:00Z", 3, null]
    ]
  }
}
```

All .json responses from ERDDAP have
- a `table` object (with name=value pairs).
- a `columnNames, columnTypes,` and `columnUnits` array, with a value for each column.
- a `rows` array of arrays with the rows and columns of data.
- `null`'s for missing values.

Once you figure out how to process one ERDDAP .json table using your preferred JSON library or toolkit, it should be easy to process all other tables from ERDDAP in a similar way.

JSONP (from .json and .geoJson) - Jsonp is an easy way for a JavaScript script on a web page to import and access data from ERDDAP. Requests for .geoJson, .json, and .ncoJson files may include an optional jsonp request by adding `&.jsonp=`*`functionName`* to the end of the query. Basically, this just tells ERDDAP to add *`functionName`*`(` to the beginning of the response and `")"` to the end of the response. The functionName must be a series of 1 or more (period-separated) words. For each word, the first character of *functionName* must be an ISO 8859 letter or "_". Each optional subsequent character must be an ISO 8859 letter, "_", or a digit. If originally there was no query, leave off the "&" in your query. After the data download to the web page has finished, the data is accessible to the JavaScript script via that JavaScript function. Here is an example using jsonp and Javascript with ERDDAP (thanks to Jenn Patterson Sevadjian of PolarWatch).

MATLAB .mat - Matlab users can use tabledap's .mat file type to download data from within MATLAB. Here is a one line example:

```
load(urlwrite('https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.mat?time,T_25&station="0n0e"&time>=2015-05-23T12:
```

(You may need to percent encode the query part of the URL.) The data will be in a MATLAB structure. The structure's name will be the datasetID (for example, `pmelTaoDySst`). The structure's internal variables will be column vectors with the same names as in ERDDAP (for example, use `fieldnames(pmelTaoDySst)`). You can then make a scatterplot of any two columns. For example:

```
plot(pmelTaoDySst.time, pmelTaoDySst.T_25)
```

ERDDAP stores datetime values in .mat files as "seconds since 1970-01-01T00:00:00Z". To display one of these values as a String in Matlab, you can use, e.g., `datastr(cwwcNDBCMet.time(1)/86400 + 719529)` 86400 converts ERDDAP's "seconds since" to Matlab's "days since". 719529 converts ERDDAP's base time of "1970-01-01T00:00:00Z" to Matlab's "0000-01-00T00:00:00Z".

.mat files have a maximum length for identifier names of 32 characters. If a variable name is longer than that, ERDDAP changes the name of the variable when writing the .mat file: it generates a hash digest of the variable name and appends that after the first 25 characters. Thus, long variable names that only differ at the end will still have unique names in the .mat file.

ERDDAP administrators: you can avoid this problem by specifying shorter variable destinationNames.

NetCDF .nc - Requests for .nc files will always return the data in a table-like, NetCDF-3, 32-bit, .nc file:

- All variables will use the file's "row" dimension.
- Since NetCDF-3 files don't support variable length strings and only support chars with 1 byte per char, all String variables in ERDDAP (variable length, 2 bytes per char) will be stored as char array variables (1 byte per char) in the .nc file, using the ISO-8859-1 character set. The char variables will have an extra dimension indicating the maximum number of characters per string. Characters not in the ISO-8859-1 character set will be converted to '?'.
- Since chars in NetCDF-3 files are only 1 byte per char, all char variables in ERDDAP (2 bytes per char) will be stored as chars (one byte per char) in the .nc file, using the ISO-8859-1 charset. Characters not in the ISO-8859-1 character set will be converted to '?'.
- Since NetCDF-3 files do not support 64-bit long integers, all long variables in ERDDAP will be stored as double variables in the .nc file. Numbers from -2^53 to 2^53 will be stored exactly. Other numbers will be stored as approximations.

Don't use NetCDF-Java, NetCDF-C, NetCDF-Fortran, NetCDF-Perl, or Ferret to try to access a remote ERDDAP .nc file. It won't work. Instead, use this approach.

.ncCF - Requests for a .ncCF file will return a version 3, 32-bit, NetCDF .nc file with the Contiguous Ragged Array Representation associated with the dataset's cdm_data_type, as defined in the CF Discrete Geometries conventions (which were previously named "CF Point Observation Conventions").

- Point - Appendix H.1 Point Data
- TimeSeries - Appendix H.2.4 Contiguous ragged array representation of timeSeries
- Profile - Appendix H.3.4 Contiguous ragged array representation of profiles
- Trajectory - Appendix H.4.3 Contiguous ragged array representation of trajectories
- TimeSeriesProfile - Appendix H.5.3 Ragged array representation of timeSeriesProfiles
- TrajectoryProfile - Appendix H.6.3 Ragged array representation of trajectoryProfiles

A request will succeed only if the dataset has a cdm_data_type other than "Other" and if the request includes at least one data variable (not just the outer, descriptive variables). The file will include longitude, latitude, time, and other required descriptive variables, even if you don't request them.

.ncCFHeader - Requests for .ncCFHeader files will return the header information (text) that would be generated if you used ncdump -h *fileName* on the corresponding .ncCF file.

.ncCFMA - Requests for a .ncCFMA file will return a version 3, 32-bit, NetCDF .nc file with the Complete or Incomplete, depending on the data, Multidimensional Array Representation associated with the dataset's cdm_data_type, as defined in the CF Discrete Sampling Geometries conventions, which were previously named "CF Point Observation Conventions". This is the file type used by the NODC Templates. A request will succeed only if the dataset has a cdm_data_type other than "Other" and if the request includes at least one data variable (not just the outer, descriptive variables). The file will include longitude, latitude, time, and other required descriptive variables, even if you don't request them.

.ncCFMAHeader - Requests for .ncCFMAHeader files will return the header information (text) that would be generated if you used ncdump -h *fileName* on the corresponding .ncCFMA file.

NetCDF-Java, NetCDF-C, NetCDF-Fortran, and NetCDF-Perl - Don't try to access an ERDDAP tabledap dataset URL directly with a NetCDF library or tool (by treating the tabledap dataset as an OPeNDAP dataset or by creating a URL with the .nc file extension). It won't work.

1. These libraries and tools just work with OPeNDAP gridded data. Tabledap offers data as OPeNDAP sequence data, which is a different data structure.
2. These libraries and tools just work with actual, static, .nc files. But an ERDDAP URL with the .nc file extension is a virtual file. It doesn't actually exist.

Fortunately, there is a two step process that does work:

1. Use the ERDDAP web pages or a program like curl to download a .nc file with a subset of the dataset.
2. Open that local .nc file (which has the data in a grid format) with one of the NetCDF libraries or tools. With NetCDF-Java for example, use:
   ```
   NetcdfFile nc =
   NetcdfFile.open("c:\downloads\theDownloadedFile.nc"); //pre v5.4.1
   NetcdfFile nc =
   NetcdfFiles.open("c:\downloads\theDownloadedFile.nc"); //v5.4.1+
   NetcdfDataset nc =
   NetcdfDataset.openDataset("c:\downloads\theDownloadedFile.nc");
   //pre v5.4.1
   ```
   or
   ```
   NetcdfDataset nc =
   NetcdfDatasets.openDataset("c:\downloads\theDownloadedFile.nc");
   //v5.4.1+
   ```
   (NetcdfFiles are a lower level approach than NetcdfDatasets. It is your choice.) In both cases, you can then do what you want with the `nc` object, for example, request metadata or request a subset of a variable's data as you would with any other .nc file.

Ocean Data View .odvTxt - ODV users can download data in a ODV Generic Spreadsheet Format .txt file by requesting tabledap's .odvTxt fileType. The selected data MUST include longitude, latitude, and time variables. Any longitude values (0 to 360, or -180 to 180) are fine. After saving the resulting file (with the extension .txt) in your computer:

1. Open ODV.

2. Use `File : Open`.
   - Change `Files of type` to `Data Files (*.txt *.csv *.jos *.o4x)`.
   - Browse to select the .txt file you created in ERDDAP and click on `Open`. The data should now be visible on a map in ODV.
3. See ODV's `Help` menu for more help using ODV.

OPeNDAP Libraries - Although ERDDAP is an OPeNDAP -compatible data server, you can't use most OPeNDAP client libraries, including NetCDF-Java, NetCDF-C, NetCDF-Fortran, NetCDF-Perl , or Ferret , to get data directly from an ERDDAP tabledap dataset because those libraries don't support the OPeNDAP Selection constraints that tabledap datasets use for requesting subsets of the dataset, nor do they support the sequence data structure in the response. (But see this other approach that works with NetCDF libraries.) But you can use the Pydap Client or Java-DAP2 , because they both support Selection constraints. With both the Pydap Client and Java-DAP2, when creating the initial connection to an ERDDAP table dataset, use the tabledap dataset's base URL, e.g.,
`https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst`

- Don't include a file extension (e.g., .nc) at the end of the dataset's name.
- Don't specify a variable or a Selection-constraint.

Once you have made the connection to the dataset, you can request metadata or a subset of a variable's data via a Selection-constraint.

Pydap Client users can access tabledap datasets via ERDDAP's standard OPeNDAP services. See the Pydap Client instructions for accessing sequential data . Note that the name of a dataset in tabledap will always be a single word, e.g., pmelTaoDySst in the OPeNDAP dataset URL
`https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst` and won't ever have a file extension (unlike, for example, .cdp for the sample dataset in the Pydap instructions). Also, the name of the sequence in tabledap datasets will always be "s" (unlike "location" for the sample dataset in the Pydap instructions).

Python is a widely-used computer language that is very popular among scientists. In addition to the Pydap Client, you can use Python to download various files from ERDDAP as you would download other files from the web:

```
import urllib
urllib.urlretrieve("https://baseUrl/erddap/tabledap/datasetID.fileType?query", "outputFileName")
```

(You may need to percent encode the query part of the URL.)
Or download the content to an object instead of a file:

```
import urllib2
response = urllib2.open("https://baseUrl/erddap/tabledap/datasetID.fileType?query")
theContent = response.read()
```

There are other ways to do this in Python. Search the web for more information.

erddapy (ERDDAP + Python, by Filipe Pires Alvarenga Fernandes) and
erddap-python (by Favio Medrano)
are Python libraries that "take advantage of ERDDAP's RESTful web services and create the ERDDAP URL for any request like searching for datasets, acquiring metadata, downloading data, etc." They have somewhat different programming styles and slightly different feature sets, so it might be good to experiment with both to see which you prefer.

"Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and explanatory text" using any of over 40 programming languages, including Python and R. Here are two sample Jupyter Notebooks that access ERDDAP using Python: ERDDAP Advanced Search Test and ERDDAP Timing . Thanks to Rich Signell.

R Statistical Package - R is an open source statistical package for many operating systems. In R, you can download a .csv file from ERDDAP and then import data from that .csv file into an R structure (e.g., `test`). For example:

```
download.file(url="https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.csv?longitude,latitude,time,station,wmo_pla
test<-read.csv(file="/home/bsimons/test.csv")
```

(You may need to percent encode the query part of the URL.)

There are third-party R packages designed to make it easier to work with ERDDAP from within R: rerddap , rerddapXtracto , and plotdap . Thanks to rOpenSci and Roy Mendelssohn.

.wav - ERDDAP can return data in .wav files, which are uncompressed audio files.

- You can save any numeric data in .wav files, but this file format is clearly intended to be used with PCM digitized sound waves. We do not recommend saving other types of data in .wav files.
- All of the data variables you select to save in a .wav file must have the same data type, e.g., int. Currently, because of a bug in Java 8, ERDDAP can't save float or double data in .wav files. That should change when ERDDAP is run with Java 9.
- The data variable values are just written as a long sequence of numbers (the sound wave). Unfortunately, ERDDAP must create the entire .wav file before it can begin to send the file to you, so there will be a delay from when you submit the request and when you start to get the file. There is no limit in ERDDAP to the number of numbers that may be written to a .wav file, but requests for very large files may lead to timeout error messages because of the long time needed to create the file.
- Data is written to .wav files using either the PCM_SIGNED encoding (for integer data types) or the PCM_FLOAT encoding (for floating point data types). Since long integer values aren't supported by PCM_SIGNED, ERDDAP writes longs as ints, by writing the high 4 bytes of the long to the file. Other than long values, data values are unchanged when they are written to .wav files.

- If the dataset's global attributes include `audioFrameRate` and/or `audioSampleRate` attributes, those values (they should be the same) will be used as the frameRate and sampleRate. Otherwise, ERDDAP assigns a rate of 10000 frames (or samples) per second.
- The .wav files will have several attributes describing how the sound data is stored in the file. Currently, no other metadata (e.g., the global attributes) is stored in the .wav files.

Making an Image File with a Graph or Map of Tabular Data If a tabledap request URL specifies a subset of data which is suitable for making a graph or a map, and the fileType is an image fileType, tabledap will return an image with a graph or map. tabledap request URLs can include optional graphics commands which let you customize the graph or map. As with other tabledap request URLs, you can create these URLs by hand or have a computer program do it. Or, you can use the Make A Graph web pages, which simplify creating these URLs (see the "graph" links in the table of tabledap datasets).

The fileType options for downloading images of graphs and maps of table data are:

Image fileTypes	Description	Info	Example
.kml	View a .kml file, suitable for Google Earth.	info	example
.largePdf	View a large .pdf image file with a graph or map.	info	example
.largePng	View a large .png image file with a graph or map.	info	example
.pdf	View a standard, medium-sized .pdf image file with a graph or map.	info	example
.png	View a standard, medium-sized .png image file with a graph or map.	info	example
.smallPdf	View a small .pdf image file with a graph or map.	info	example
.smallPng	View a small .png image file with a graph or map.	info	example
.transparentPng	View a .png image file (just the data, without axes, landmask, or legend).	info	example

Image Size - ".small" and ".large" were ERDDAP's original system for making different-sized images. Now, for .png and .transparentPng images (not other image file types), you can also use the &.size=*width*|*height* parameter to request an image of any size.

.transparentPng - The .transparentPng file type will make a graph or map without the graph axes, landmask, or legend, and with a transparent (not opaque white) background. This file type can be used for any type of graph or map. For graphs and maps, the default size is 360x360 pixels. Or, you can use the &.size=*width*|*height* parameter to request an image of any size.

Incompatibilities
Some results file types have restrictions. For example, Google Earth .kml is only appropriate for results with longitude and latitude values. If a given request is incompatible with the requested file type, tabledap throws an error.

Command Line Downloads with curl
If you want to download a series of files from ERDDAP, you don't have to request each file's ERDDAP URL in your browser, sitting and waiting for each file to download. If you are comfortable writing computer programs (e.g., with C, Java, Python, Matlab, r) or scripts (e.g., Python, bash, tcsh, PowerShell, or Windows batch files), you can write a program or script with a loop (or a series of commands) that imports all of the desired data files. Or, if you are comfortable running command line programs (from a Linux or Windows command line, or a Mac OS Terminal), you can use curl (or a similar program like wget) to save results files from ERDDAP into files on your hard drive, without using a browser or writing a computer program or script.

ERDDAP + curl is amazingly powerful and allows you to use ERDDAP in many new ways.
On Linux and Mac OS X, curl is probably already installed.
On Mac OS X, to get to a command line, use "Finder : Go : Utilities : Terminal".
On Windows, you need to download curl (the "Windows 64 - binary, the curl project" variant worked for me on Windows 10) and install it.
On Windows, to get to a command line, click on "Start" and type "cmd" into the search text field.
Please be kind to other ERDDAP users: run just one script or curl command at a time.
Instructions for using curl are on the curl man page and in this curl tutorial.
But here is a quick tutorial related to using curl with ERDDAP:

- To download and save one file, use
  `curl --compressed -g "erddapUrl" -o fileDir/fileName.ext`
  where `--compressed` tells curl to request a compressed response and automatically decompress it,
  `-g` disables curl's globbing feature,
  `erddapUrl` is any ERDDAP URL that requests a data or image file, and
  `-o fileDir/fileName.ext` specifies the name for the file that will be created.
  For example,

  ```
  curl --compressed -g "https://coastwatch.pfeg.noaa.gov/erddap/tabledap/cwwcNDBCMet.png?time,atmp&time%3E=2010-09-03T00:00
  ```

  In curl, as in many other programs, the query part of the erddapUrl must be percent encoded. To do this, you need to convert special characters (other than the initial '&' and the main '=') in all constraints into the form %HH, where HH is the 2 digit hexadecimal value of the character. Usually, you just need to convert a few of the punctuation characters: % into %25, & into %26, " into %22, < into %3C, = into %3D, > into %3E, + into %2B, | into %7C, space into %20, [into %5B,] into %5D, and convert all characters above #127 into their UTF-8 bytes and then percent encode each byte of the UTF-8 form into the %HH format (ask a programmer for help). For example, `&stationID>="41004"` becomes `&stationID%3E=%2241004%22` Note that percent encoding is generally

required when you access ERDDAP via software other than a browser. Browsers usually handle percent encoding for you. In some situations, you need to percent encode all characters other than A-Za-z0-9_-!.~'()* . Programming languages have tools to do this (for example, see Java's java.net.URLEncoder and JavaScript's encodeURIComponent()) and there are websites that percent encode/decode for you .

- To download and save many files in one step, use curl with the globbing feature enabled: `curl --compressed "erddapUrl" -o fileDir/fileName#1.ext`
  Since the globbing feature treats the characters [,], {, and } as special, you must also percent encode them in the erddapURL as %5B, %5D, %7B, %7D, respectively. Fortunately, these are rare in tabledap URLs. Then, in the erddapUrl, replace a zero-padded number (for example `01`) with a range of values (for example, `[01-15]`), or replace a substring (for example `TAML1`) with a list of values (for example, `{TAML1,41009,46088}`). The `#1` within the output fileName causes the current value of the range or list to be put into the output fileName. For example,

  ```
  curl --compressed "https://coastwatch.pfeg.noaa.gov/erddap/tabledap/cwwcNDBCMet.png?time,atmp&time%3E=2010-09-03T00:00:00
  ```

  (That example includes `--compressed` because it is a generally useful option, but there is little benefit to `--compressed` when requesting .png files because they are already compressed internally.)
- To access a password-protected, private ERDDAP dataset with curl via https, use this two-step process:

  1. Log in (authenticate) and save the certificate cookie in a cookie-jar file:

     ```
     curl -v --data 'user=myUserName&password=myPassword' -c cookies.txt -b cookies.txt -k https://baseurl:8443/erddap/log
     ```

  2. Repeatedly make data requests using the saved cookie:

     ```
     curl --compressed -v -c cookies.txt -b cookies.txt -k https://baseurl:8443/erddap/tabledap/datasetID.fileType?query -
     ```

  Some ERDDAP installations won't need the port number (:8443) in the URL.
  (Thanks to Liquid Robotics for the starting point and Emilio Mayorga of NANOOS and Paul Janecek of Spyglass Technologies for testing.)

- **query** is the part of the request after the "?". It specifies the subset of data that you want to receive. In tabledap, it is an OPeNDAP DAP selection constraint query in the form: `{resultsVariables}{constraints}` . For example, `longitude,latitude,time,station,wmo_platform_code,T_25&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z`
  - ***resultsVariables*** is an optional comma-separated list of variables (for example, `longitude,latitude,time,station,wmo_platform_code,T_25`).
    For each variable in resultsVariables, there will be a column in the results table, in the same order. If you don't specify any results variables, the results table will include columns for all of the variables in the dataset.
  - ***constraints*** is an optional list of constraints, each preceded by & (for example, `&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z`).
    - The constraints determine which rows of data from the original table table are included in the results table. The constraints are applied to each row of the original table. If all the constraints evaluate to `true` for a given row, that row is included in the results table. Thus, "&" can be roughly interpreted as "and".
    - If you don't specify any constraints, all rows from the original table will be included in the results table. For the fileTypes that return information about the dataset (notably, .das, .dds, and .html), but don't return actual data, it is fine not to specify constraints. For example, `https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.dds`
      For the fileTypes that do return data, not specifying any constraints may result in a very large results table.
    - tabledap constraints are consistent with OPeNDAP DAP selection constraints , but with a few additional features.
    - Each constraint is in the form `VariableOperatorValue` (for example, `latitude>45`).
    - The valid operators are =, != (not equals), =~ (a regular expression test), <, <=, >, and >= .
    - tabledap extends the OPeNDAP standard to allow any operator to be used with any data type. (Standard OPeNDAP selections don't allow <, <=, >, or >= to be used with string variables and don't allow =~ to be used with numeric variables.)
      - For String variables, all operators act in a case-sensitive manner.
      - For String variables, queries for ="" and !="" should work correctly for almost all datasets.
      - For all constraints of String variables and for regex constraints of numeric variables, the right-hand-side value MUST be enclosed in double quotes (e.g., `id="NDBC41201"`) and any internal special characters must be backslash encoded: \ into \\, " into \", newline into \n, and tab into \t.
      - For all constraints, the value part of the `VariableOperatorValue` MUST be percent encoded : special characters in the query values (the parts after the '=' signs) are encoded as %HH, where HH is the 2 digit hexadecimal value of the character (for example, ' ' is replaced with "%20"). Characters above #127 must be converted to UTF-8 bytes, then each UTF-8 byte must be percent encoded. Normally, when you use a browser, the browser takes care of this for you. But if your computer program or script generates the URLs, it probably needs to do the percent encoding itself. If so, you need to encode all characters other than A-Za-z0-9_-!.~'()* in all query values. Programming languages have tools to do this (for example, see Java's java.net.URLEncoder and JavaScript's encodeURIComponent()) and there are websites that percent encode/decode for you .

- WARNING: Numeric queries involving =, !=, <=, or >= may not work as desired with floating point numbers. For example, a search for `longitude=220.2` may fail if the value is stored as 220.20000000000001. This problem arises because floating point numbers are not represented exactly within computers. When ERDDAP performs these tests, it allows for minor variations and tries to avoid the problem. But it is possible that some datasets will still have problems with these queries and return unexpected and incorrect results.
- NaN (Not-a-Number) - Many numeric variables have an attribute which identifies a number (e.g., -99) as a missing_value or a _FillValue. When ERDDAP tests constraints, it always treats these values as NaN's. So:
  Don't create constraints like `temperature!=-99`.
  Do create constraints like `temperature!=NaN`.
  - For numeric variables, tests of *`variable`*`=NaN` (Not-a-Number) and *`variable`*`!=NaN` will usually work as expected.
    WARNING: numeric queries with =NaN and !=NaN may not work as desired since many data sources don't offer native support for these queries and ERDDAP can't always work around this problem. For some datasets, queries with =NaN or !=NaN may fail with an error message (insufficient memory or timeout) or erroneously report `Your query produced no matching results.`
  - For numeric variables, tests of *`variable`*`<NaN` (Not-a-Number) (or <=, >, >=) will return false for any value of the variable, even NaN. NaN isn't a number so these tests are nonsensical. Similarly, tests of *`variable<aNonNaNValue`* (or <=, >, >=) will return false whenever the variable's value is NaN.
- *variable*=~"*regularExpression*" tests if the value from the variable on the left matches the regular expression on the right.
  - tabledap uses the same regular expression syntax (tutorial) as is used by Java.
  - A common use of regular expressions is to provide the equivalent of an OR constraint, which DAP doesn't support. For example, if you want to request data from station="Able" OR station="Baker" OR station="Charlie", you can use a capture group in a regular expression (which is the =~ operator in DAP constraints). A capture group is surrounded by parentheses. The options within a capture group are separated by "|", which you can read/interpret as "OR". Since the constraint is a string, it must be surrounded by quotes. So the regular expression constraint for those stations is:
    `&station=~"(Able|Baker|Charlie)"`
  - **WARNING** - For numeric variables, the =~ test is performed on the String representation of the variable's source values. So if the variable's source uses scale_factor and/or add_offset metadata (which ERDDAP uses to unpack the data) or if the variable is an altitude variable that ERDDAP has converted from a depth value, the =~ test will be applied to the raw source values. This is not a good situation, but there is no easy way to deal with it. It is fine to try =~ tests of numeric variables, but be very skeptical of the results.
  - **WARNING: queries with =~ may not work as desired** since many data sources don't offer native support for these queries and ERDDAP can't always work around this problem. ERDDAP sometimes has to get the entire dataset to do the test itself. For some datasets, queries with =~ may fail with an error message (insufficient memory or timeout) or erroneously report `Your query produced no matching results..`

- tabledap extends the OPeNDAP standard to allow constraints (selections) to be applied to any variable in the dataset. The constraint variables don't have to be included in the resultsVariables. (Standard OPeNDAP implementations usually only allow constraints to be applied to certain variables. The limitations vary with different implementations.)
- Although tabledap extends the OPeNDAP selection standard (as noted above), these extensions (notably "=~" being applicable to numeric variables) sometimes aren't practical because tabledap may need to download lots of extra data from the source (which takes time) in order to test the constraint.
- tabledap always stores date/time values as double precision floating point numbers (seconds since 1970-01-01T00:00:00Z, sometimes with some number of milliseconds). Here is an example of a query which includes date/time numbers:
  `https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.htmlTable?longitude,latitude,time,station,wmo_platform_code,T_25&time>=1432382400&time<=1433073600`
  The more human-oriented fileTypes (notably, .csv, .tsv, .htmlTable, .odvTxt, and .xhtml) display date/time values as ISO 8601:2004 "extended" date/time strings (e.g., `2002-08-03T12:30:00Z`, but some time variables in some datasets include milliseconds, e.g., `2002-08-03T12:30:00.123Z`).
  ERDDAP has a utility to Convert a Numeric Time to/from a String Time.
  See also: How ERDDAP Deals with Time.
- tabledap extends the OPeNDAP standard to allow you to specify time values in the ISO 8601 date/time format (*`YYYY-MM-DDThh:mm:ss.SSSZ`*, where Z is 'Z' or a ±hh or ±hh:mm offset from the Zulu/GMT time zone. If you omit Z and the offset, the Zulu/GMT time zone is used. Separately, if you omit .SSS, :ss.SSS, :mm:ss.SSS, or Thh:mm:ss.SSS from the ISO date/time that you specify, the missing fields are assumed to be 0. In some places, ERDDAP accepts a comma (ss,SSS) as the seconds decimal point, but ERDDAP always uses a period when formatting times as ISO 8601 strings. Here is an example of a query which includes ISO date/time values:
  `https://coastwatch.pfeg.noaa.gov/erddap/tabledap`

/pmelTaoDySst.htmlTable?longitude,latitude,time,station,wmo_platform_code,T_25&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z
ERDDAP is "lenient" when it parses date/time strings. That means that date/times with the correct format, but with month, date, hour, minute, and/or second values that are too large or too small will be rolled to the appropriate date/times. For example, ERDDAP interprets 2001-12-32 as 2002-01-01, and interprets 2002-01-00 as 2001-12-31. (It's not a bug, it's a feature! We understand that you may object to this if you are not familiar with lenient parsing. We understand there are circumstances where some people would prefer strict parsing, but there are also circumstances where some people would prefer lenient parsing. ERDDAP can't have it both ways. This was a conscious choice. Lenient parsing is the default behavior in Java, the language that ERDDAP is written in and arguably the most-used computer language. Also, this behavior is consistent with ERDDAP's conversion of requested grid axis values to the nearest valid grid axis value. And this is consistent with some other places in ERDDAP that try to repair invalid input when the intention is clear, instead of just returning an error message.)
ERDDAP has a utility to Convert a Numeric Time to/from a String Time.
See also: How ERDDAP Deals with Time.

- tabledap extends the OPeNDAP standard to allow you to specify constraints for time and timestamp variables relative to `now`. The constraint can be simply, for example, `time<now`, but usually the constraint is in the form
  `now(+| |-)`*`positiveInteger`*`(second|seconds|minute|minutes|hour|hours|day|days|month|months|year|years)`
  for example, `now-7days`.
  Months and years are interpreted as calendar months and years (not UDUNITS-like constant values). This feature is especially useful when creating a URL for an <img> (image) tag on a web page, since it allows you to specify that the image will always show, for example, the last 7 days worth of data (`&time>now-7days`).
  Note that a '+' in the URL is percent-decoded as ' ', so you should percent-encode '+' as %2B. However, ERDDAP interprets `"now "` as `"now+"`, so in practice you don't have to percent-encode it.
- tabledap extends the OPeNDAP standard to allow you to refer to `min(`*`variableName`*`)` or `max(`*`variableName`*`)` in the right hand part of a constraint.
  - If *variableName* is a timestamp variable, the constraint must be in the form:
    `min|max(`*`variableName`*`)[+|-`*`positiveInteger`*`[millis|seconds|minutes|hours|days|months|years]]`
    (or singular units). If time units aren't supplied, "seconds" are assumed. For example, `&time>max(time)-10minutes`
  - If *variableName* is a non-timestamp variable, the constraint must be in the form:
    `min|max(`*`variableName`*`)[+|-`*`positiveNumber`*`]`
    For example, `&pressure<min(pressure)+10.5`
  - Usually, *variableName* will be the same as the variable you are constraining, but it doesn't have to be.
  - If *variableName*'s minimum (or maximum) isn't known, min() (or max()) will throw an error.
  - min() and max() will almost always be used to constrain numeric (including timestamp) variables, but it is technically possible to constrain string variables.
  - min() and max() can't be used with regex constraints (=~).

- Can users extract data from a polygon-defined geographic area?
  No. ERDDAP does not support that. You need to calculate the lat lon bounding box for the polygon (it is easy to calculate: the lon range will be minLon to maxLon, the lat range will be minLat to maxLat) and request that data from ERDDAP using the constraint system described above. You can then do the polygon extraction (or calculations) in your favorite analysis software (ArcGIS, Matlab, Python, R, ...). An advantage of this approach is that there is no limit to the number of points that define the polygon. Note that this is the way the OPeNDAP standard (which ERDDAP uses for these requests) works: users request a subset of the original data based on simple constraints.

- **Server-side Functions** - The OPeNDAP standard supports the idea of server-side functions, but doesn't define any. TableDAP supports some server-side functions that modify the results table before the results table is sent to you. Server-side functions are OPTIONAL. Other than 'distinct', they are rarely needed. Currently, each of the functions takes the results table as input and returns a table with the same columns in the same order (but the rows may be altered). If you use more than one function, they will be applied in the order that they appear in the request. Most of these options appear near the bottom of the Data Access Form and Make A Graph web page for each dataset.

  - `&addVariablesWhere("`*`attributeName`*`","`*`attributeValue`*`")`
    is different than the other server-side functions in that it doesn't filter the results table. Instead, if you add it to a query, ERDDAP will add all of the variables in the dataset which have *`attributeName=attributeValue`* to the list of requested variables. For example, if you add `&addVariablesWhere("ioos_category","Wind")` to a query, ERDDAP will add all of the variables in the dataset that have an `ioos_category=Wind` attribute to the list of requested variables (for example, windSpeed, windDirection, windGustSpeed). *attributeName* and *attributeValue* are case-sensitive. You can specify 0 or more `&addVariablesWhere()` functions to the request.

  - `&distinct()`
    If you add `&distinct()` to the end of a query, ERDDAP will sort all of the rows in the results table (starting with the first requested variable, then using the second

requested variable if the first variable has a tie, ...), then remove all non-unique rows of data. For example, the query `stationType,stationID&distinct()` will return a sorted list of stationIDs associated with each stationType. In many situations, ERDDAP can return distinct() values quickly and efficiently. But in some cases, ERDDAP must look through all rows of the source dataset. If the data set isn't local, this may be VERY slow, may return only some of the results (without an error), or may throw an error.

- `&orderBy("`*`comma-separated list of variable names`*`")`
  The comma-separated (CSV) list of 1 or more variable names lets you specify how the results table will be sorted (starting with the first variable in the CSV list, then using the second variable if the first variable has a tie, ...). All of the orderBy variables MUST be included in the list of requested variables in the query as well as in the orderBy CSV list of variables (the .html and .graph forms will do this for you).

  Without orderBy, the rows of data in the response table are in the order they arrived from the data source, which may or may not be a nice logical order. Thus, orderBy allows you to request that the results table be sorted in a specific way. For example, use the query

  `?stationID,time,temperature&time>2026-04-04&orderBy("stationID,time")`

  to get the results sorted by stationID, then time. Or use the query

  `?stationID,time,temperature&time>2026-04-04&orderBy("time,stationID")`

  to get the results sorted by time first, then stationID.

  `orderBy()` doesn't support the divisor options for numeric variables, i.e., `numericVariable[/number[timeUnits][:offset]]` because they would be nonsensical.

- `&orderByDescending("`*`comma-separated list of variable names`*`")`
  orderByDescending is just like orderBy, except the rows are sorted in descending, instead of ascending, order.
- `&orderByClosest("`*`comma-separated list of variable names`*`")`
  The comma-separated (CSV) list of 1 or more variable names lets you specify how the results table will be sorted. For orderByClosest, the last item in the CSV list must be a numeric variable with a divisor. (See divisor details.) Other variables in the CSV list may not have divisors.
  All of the orderByClosest variables MUST be included in the list of requested variables in the query as well as in the orderByClosest CSV list of variables (the .html and .graph forms will do this for you).

  For orderByClosest, for each group, ERDDAP will return just the row where the value of the last CSV variable is closest to the divisor (interval). For example,

  `?stationID,time,temperature&time>2026-04-04&orderByClosest("stationID,time/2hours")`

  will sort by stationID and time, but only return the rows for each stationID where the last orderBy column (time) are closest to 2hour intervals (12am, 2am, 4am, ...). For numeric variables in orderByMax CSV list, for each group, ERDDAP will return the exact value (e.g., the exact time) at which the closest value occurred. This is the closest thing in tabledap to stride values in a griddap request.

- `&orderByCount("`*`comma-separated list of variable names`*`")`
  The comma-separated (CSV) list of 0 or more variable names lets you specify how the results table will be sorted and grouped. Numeric variables in the CSV list usually have an optional divisor to identify groupings, e.g., time/1day . (See divisor details.)
  All of the orderByCount variables MUST be included in the list of requested variables in the query as well as in the orderByCount CSV list of variables (the .html and .graph forms will do this for you).

  For orderByCount, for each group, ERDDAP will return just one row with the count of the number of non-NaN values for each variable not in the CSV list. For example, use the query

  `?stationID,time,temperature,windspeed&time>2026-04-04&orderByCount("stationID,time/1day")`

  to get a count of the number of non-NaN temperature and windspeed values for each stationID, for each day (for stations with data from after 2026-04-04).

  Divisors - All orderBy options (other than the plain `orderBy()` and `orderByClosest()`) support divisor options for any of the numeric variables in the orderBy... CSV list, in the form `numericVariable[/number[timeUnits][:offset]]`
  Some examples are: time/10, time/2days, depth/10, depth/0.5, depth/10:5.

  - The divisor number must be a positive value. If timeUnits aren't month or year, the number may be a floating point number. The number is interpreted to have the same units as the variable; for timestamp variables, unless modified by timeUnits, the number is interpreted as units=seconds.
  - The optional timeUnits is only allowed if the numericVariable is a timestamp variable. All common English time units are supported (singular or plural) and with many abbreviations:
    ms, msec, msecs, millis, millisec, millisecs, millisecond, milliseconds,
    s, sec, secs, second, seconds, m, min, mins, minute, minutes, h, hr, hrs, hour, hours,
    d, day, days, week, weeks, mon, mons, month, months, yr, yrs, year, or years.

- The optional :offset is only allowed if timeUnits isn't specified.
- If numericVariable is a timestamp variable and the timeUnits are months or years, the number must be a positive integer and ERDDAP will increment by calendar months or years (not a fixed length of time). If timeUnits is month, number must be 1, 2, 3, 4, or 6.

- `&orderByLimit("comma-separated list of variable names")`
  The comma-separated (CSV) list of 0 or more variable names plus a limit number lets you specify how the results table will be sorted and grouped. Numeric variables in the CSV list usually have an optional divisor to identify groupings, e.g., time/1day . (See divisor details.)
  All of the orderByLimit variables MUST be included in the list of requested variables in the query as well as in the orderByLimit CSV list of variables (the .html and .graph forms will do this for you).

  For orderByLimit, the last value in the CSV list must be the limit number (e.g., 10). Within each sort group, only the first 'limit' rows will be kept. For example,

  ```
  ?stationID,time,temperature&time>2026-04-04&orderByLimit("stationID,time/1day,10")
  ```

  will sort by stationID and time, but only return the first 10 rows for each stationID per day. This will usually return the same rows as the first n rows per group of a similar request with no orderByLimit, but not always. This is similar to SQL's LIMIT clause.

- `&orderByMax("comma-separated list of variable names")`
  The comma-separated (CSV) list of 1 or more variable names lets you specify how the results table will be sorted and grouped. Numeric variables in the CSV list usually have an optional divisor to identify groupings, e.g., time/1day . (See divisor details.)
  For orderByMax, the last variable name in the CSV list may be a string or a numeric variable (if numeric, it must not include a divisor).
  All of the orderByMax variables MUST be included in the list of requested variables in the query as well as in the orderByMax CSV list of variables (the .html and .graph forms will do this for you).

  orderByMax will sort results into groups based on values of the variables in the CSV list except the last variable, then just keep the row within each group where the last CSV list variable has the highest value. (If there are two or more rows which have the same highest value, ERDDAP may return any of them.) For example, use the query

  ```
  ?stationID,time,temperature&time>2026-04-04&orderByMax("stationID,time/1day,temperature")
  ```

  to get just the rows of data with each station's maximum temperature value for each day (for stations with data from after 2026-04-04). For numeric variables in orderByMax CSV list, for each group, ERDDAP will return the exact value (e.g., the exact time) at which the max value (e.g., temperature) occurred. This is the closest thing in tabledap to griddap's allowing requests for the `[last]` axis value.

- `&orderByMin("comma-separated list of variable names")`
  orderByMin is exactly like orderByMax, except that it returns the row within each group which has the minimum value of the last variable in the CSV list.

- `&orderByMinMax("comma-separated list of 1 or more variable names")`
  orderByMinMax is like orderByMax, except that it returns the two rows within each group: the row which has the minimum value of the last variable in the CSV list and the row which has the maximum value of the last variable in the CSV list. For example, use the query

  ```
  ?stationID,time,temperature&time>2026-04-04&orderByMinMax("stationID,time/1day,temperature")
  ```

  to get just the rows of data with each station's minimum temperature value and each station's maximum time value for each day (for stations with data from after 2026-04-04). If there is only one row of data for a given combination (e.g., stationID), there will still be two rows in the output (with identical data).

- `&orderByMean("comma-separated list of variable names")`
  The comma-separated (CSV) list of 0 or more variable names lets you specify how the results table will be sorted and grouped. Numeric variables in the CSV list usually have an optional divisor to identify groupings, e.g., time/1day . (See divisor details.)
  All of the orderByMean variables MUST be included in the list of requested variables in the query as well as in the orderByMean CSV list of variables (the .html and .graph forms will do this for you).

  For orderByMean, for each group, ERDDAP will return the mean of each of the variables not in the CSV list. You can use the same divisor options as other orderBy options (e.g., time/1day or depth/10). For example,

  ```
  ?stationID,time,temperature&time>2026-04-04&orderByMean("stationID,time/1day")
  ```

  will sort by stationID and time, but only return the mean temperature value for each stationID for each day.
  - The mean values are returned as doubles, with _FillValue=NaN used to represent empty cells (groups with no finite values for a given variable).
  - If a column has degree-related units, the mean is calculated differently: by converting the angle to x and y components, calculating the means of the x and y components, and then converting those back to the mean degrees value. If the column's units are degree_true (or a variant), the returned mean

will be in the range 0 to 360. For other degree units (e.g., degree_east, degree_north), the returned mean will be in the range -180 to 180.
    - Any requested string or character variables not in the orderByMean list are discarded (unless the value does not vary) since asking for the mean of a string variable is meaningless.

- `&orderBySum("comma-separated list of variable names")`
  The comma-separated (CSV) list of 0 or more variable names lets you specify how the results table will be sorted and grouped. Numeric variables in the CSV list usually have an optional divisor to identify groupings, e.g., time/1day . (See divisor details.)
  All of the orderBySum variables MUST be included in the list of requested variables in the query as well as in the orderBySum CSV list of variables (the .html and .graph forms will do this for you).

  For orderBySum, for each group, ERDDAP will return the sum of each of the variables not in the CSV list. You can use the same divisor options as other orderBy options (e.g., time/1day or depth/10). For example,

  `?stationID,time,rainfall&time>2026-04-04&orderBySum("stationID,time/1day")`

  will sort by stationID and time, but only return the sum of rainfall values for each stationID for each day.
    - The sum values are returned as doubles, with _FillValue=NaN used to represent empty cells (groups with no finite values for a given variable).
    - This will calculate the sums for all numeric variables. For many types of data (e.g., time variables or variables with degree-related units), the calculated sums won't make any scientific sense. It is up to you to determine the appropriateness of using the calculated values.
    - Any requested string or character variables not in the orderBySum list are discarded (unless the value does not vary) since asking for the sum of a string variable is meaningless.

- `&units("value")`
  If you add `&units("UDUNITS")` to the end of a query, the units will be described via the UDUNITS standard (for example, `degree_C`).
  If you add `&units("UCUM")` to the end of a query, the units will be described via the UCUM standard (for example, `Cel`).
  On this ERDDAP, the default for most/all datasets is UDUNITS.
  See also ERDDAP's units converter.

- **Graphics Commands** - tabledap extends the OPeNDAP standard by allowing graphics commands in the query. The Make A Graph web pages simplify the creation of URLs with these graphics commands (see the "graph" links in the table of tabledap datasets). So we recommend using the Make A Graph web pages to generate URLs, and then, when needed, using the information here to modify the URLs for special purposes. These commands are optional. If present, they must occur after the data request part of the query. These commands are used by tabledap if you request an image fileType (.png or .pdf) and are ignored if you request a data file (e.g., .asc). If relevant commands are not included in your request, tabledap uses the defaults and tries its best to generate a reasonable graph or map. All of the commands are in the form &*commandName=value* . If the value has sub-values, they are separated by the '|' character. The commands are:
  - `&.bgColor=bgColor`
    This specifies the background color of a graph (not a map). The color is specified as an 8 digit hexadecimal value in the form 0x*AARRGGBB*, where AA, RR, GG, and BB are the opacity, red, green and blue components, respectively. The canvas is always opaque white, so a (semi-)transparent graph background color blends into the white canvas. The color value string is not case sensitive. For example, a fully opaque (ff) greenish-blue color with red=22, green=88, blue=ee would be 0xff2288ee. Opaque white is 0xffffffff. Opaque light blue is 0xffccccff. The default on this ERDDAP is 0xffccccff.
  - `&.colorBar=palette|continuous|scale|min|max|nSections`
    This specifies the settings for a color bar. The sub-values are:
    - *palette* - All ERDDAP installations support a standard set of palettes: BlackBlueWhite, BlackRedWhite, BlackWhite, BlueWhiteRed, LightRainbow, Ocean, OceanDepth, Rainbow, RedWhiteBlue, ReverseRainbow, Topography, TopographyDepth, WhiteBlack, WhiteBlueBlack, WhiteRedBlack. Some ERDDAP installations support additional options. See a Make A Graph web page for a complete list. The default (no value) varies based on min and max: if -1*min ~= max, the default is BlueWhiteRed; otherwise, the default is Rainbow.
    - *continuous* - must be either no value (the default), 'C' (for Continuous), or 'D' (for Discrete). The default is different for different datasets.
    - *scale* - must be either no value (the default), `Linear`, or `Log`. The default is different for different datasets.
    - *min* - The minimum value for the color bar. The default is different for different datasets.
    - *max* - The maximum value for the color bar. The default is different for different datasets.
    - *nSections* - The preferred number of sections (for Log color bars, this is a minimum value). The default is different for different datasets.

    If you don't specify one of the sub-values, the default for the sub-value will be used.
  - `&.color=value`
    This specifies the color for data lines, markers, vectors, etc. The value must be specified as an 0xRRGGBB value (e.g., 0xFF0000 is red, 0x00FF00 is green). The color value string is not case sensitive. The default is 0x000000 (black).

- `&.draw=value`
  This specifies how the data will be drawn, as `lines`, `linesAndMarkers`, `markers` (default), `sticks`, or `vectors`.
- `&.font=scaleFactor`
  This specifies a scale factor for the font (e.g., 1.5 would make the font 1.5 times as big as normal).
- `&.land=value`
  This is only relevant if the first two results variables are longitude and latitude, so that the graph is a map. The default is different for different datasets (under the ERDDAP administrator's control via a drawLandMask in datasets.xml, or via the fallback `drawLandMask` setting in setup.xml).
  `over` makes the land mask on a map visible (land appears as a uniform gray area).
  `under` makes the land mask invisible (topography information is displayed for ocean and land areas).
  `outline` just draws the landmask outline, political boundaries, lakes, and rivers.
  `off` doesn't draw anything.
- `&.legend=value`
  This specifies whether the legend on .png images (not .pdf's) should be at the `Bottom` (default), `Off`, or `Only` (which returns only the legend).
- `&.marker=markerType|markerSize`
  markerType is an integer: 0=None, 1=Plus, 2=X, 3=Dot, 4=Square, 5=Filled Square (default), 6=Circle, 7=Filled Circle, 8=Up Triangle, 9=Filled Up Triangle, 10=Borderless Filled Square, 11=Borderless Filled Circle, 12=Borderless Filled Up Triangle. markerSize is an integer from 3 to 50 (default=5)
- `&.size=width|height`
  For .png images (not .pdf's), this specifies the desired size of the image, in pixels. This allows you to specify sizes other than the predefined sizes of .smallPng, .png, and .largePng.
- `&.trim=trimPixels`
  For .png images (not .pdf's), this tells ERDDAP to make the image shorter by removing all whitespace at the bottom, except for *trimPixels*.
- There is no `&.vars=` command. Instead, the results variables from the main part of the query are used. The meaning associated with each position varies with the `&.draw` value:
  - for lines: xAxis,yAxis
  - for linesAndMarkers: xAxis,yAxis,Color
  - for markers: xAxis,yAxis,Color
  - for sticks: xAxis,uComponent,vComponent
  - for vectors: xAxis,yAxis,uComponent,vComponent

  If xAxis=longitude and yAxis=latitude, you get a map; otherwise, you get a graph.
- `&.vec=value`
  This specifies the data vector length (in data units) to be scaled to the size of the sample vector in the legend. The default varies based on the data.
- `&.xRange=min|max|ascending|scale`
  `&.yRange=min|max|ascending|scale`
  The .xRange and .yRange commands can have any number of values.
  If there are fewer than 4 values, the missing values are treated as default values.
  If there are more than 4 values, the extra values are ignored.
  - `min` and `max` specify the range of the X and Y axes. They can be numeric values or nothing (the default, which tells ERDDAP to use an appropriate value based on the data).
  - *ascending* specifies whether the axis is drawn the normal way (ascending), or reversed (descending, flipped). *ascending* can be nothing or `true, t, false,` or `f`. These values are case insensitive. The default is `true`. *ascending* applies to graphs only, not maps.
  - *scale* specifies a preference for the axis scale. *scale* can be nothing (the default, which is dependent on the x or y axis variable's colorBarScale setting), or Linear, or Log. These values are case insensitive. *scale* applies to graphs only, not maps. *scale* will be ignored (and the axis will be linear) if the axis is a time axis, or if the minimum value on the axis is <=0, or if the axis range is very narrow.

A sample URL to view a **.png** of a **graph** is
https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.png?time,T_25&station="0n0e"&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z&.draw=lines

Or, if you change the fileType in the URL from .png to **.graph**, you can see a Make A Graph web page with that request loaded:
https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.graph?time,T_25&station="0n0e"&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z&.draw=lines
That makes it easy for humans to modify an image request to make a similar graph or map.

Or, if you change the fileType in the URL from .png to a data fileType (e.g., **.htmlTable**), you can view or download the data that was graphed:
https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.htmlTable?time,T_25&station="0n0e"&time>=2015-05-23T12:00:00Z&time<=2015-05-31T12:00:00Z&.draw=lines

A sample URL to view a **.png** of a **map** is
https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.png?longitude,latitude,T_25&time=2015-05-31T12:00:00Z&.draw=markers&.marker=5|5

Or, if you change the fileType in the URL from .png to **.graph**, you can see a Make A Graph web page with that request loaded:
https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.graph?longitude,latitude,T_25&time=2015-05-31T12:00:00Z&.draw=markers&.marker=5|5

Or, if you change the fileType in the URL from .png to a data fileType (e.g., **.htmlTable**), you can view or download the data that was mapped:
https://coastwatch.pfeg.noaa.gov/erddap/tabledap/pmelTaoDySst.htmlTable?longitude,latitude,T_25&time=2015-05-31T12:00:00Z&.draw=markers&.marker=5|5

Other Information

- **Log in to access private datasets.**
  Many ERDDAP installations don't have authentication enabled and thus don't provide any way for users to login, nor do they have any private datasets.

  Some ERDDAP installations do have authentication enabled. Currently, ERDDAP only supports authentication via Google-managed email accounts, which includes email accounts at NOAA and many universities. If an ERDDAP has authentication enabled, anyone with a Google-managed email account can log in, but they will only have access to the private datasets that the ERDDAP administrator has explicitly authorized them to access. For instructions on logging into ERDDAP from a browser or via a script, see Access to Private Datasets in ERDDAP.

- **Data Model**
  Each tabledap dataset can be represented as a table with one or more rows and one or more columns of data.
  - Each column is also known as a "data variable" (or just a "variable"). Each data variable has data of one data type. Each data variable has a name composed of a letter (A-Z, a-z) and then 0 or more characters (A-Z, a-z, 0-9, _). Each data variable has metadata which is a set of Key=Value pairs. The maximum number of columns is 2147483647, but datasets with greater than about 100 columns will be awkward for users.
  - The maximum number of rows for a dataset is probably about 9e18. In most cases, individual sources within a dataset (e.g., files) are limited to 2147483647 rows.
  - Any cell in the table may have no value (i.e., a missing value).
  - Each dataset has global metadata which is a set of Key=Value pairs.
  - Note about metadata: each variable's metadata and the global metadata is a set of 0 or more Key=Value pairs.
    - Each Key is a String consisting of a letter (A-Z, a-z) and then 0 or more other characters (A-Z, a-z, 0-9, '_').
    - Each Value is an array of one (usually) or more values of one data type.

- **Special Variables**
  ERDDAP is aware of the spatial and temporal features of each dataset. The longitude, latitude, altitude, depth, and time axis variables (when present) always have specific names and units. This makes it easier for you to identify datasets with relevant data, to request spatial and temporal subsets of the data, to make images with maps or time-series, and to save data in geo-referenced file types (e.g., .esriAscii and .kml).
  - In tabledap, a longitude variable (if present) always has the name "longitude" and the units "degrees_east".
  - In tabledap, a latitude variable (if present) always has the name "latitude" and the units "degrees_north".
  - In tabledap, an altitude variable (if present) always has the name "altitude" and the units "m" above sea level. Locations below sea level have negative altitude values.
  - In tabledap, a depth variable (if present) always has the name "depth" and the units "m" below sea level. Locations below sea level have positive depth values.
  - In tabledap, a time variable (if present) always has the name "time" and the units "seconds since 1970-01-01T00:00:00Z". If you request data and specify a time constraint, you can specify the time as a number (seconds since 1970-01-01T00:00:00Z) or as a String value (e.g., "2002-12-25T07:00:00Z" in the UTC/GMT/Zulu time zone).
  - In tabledap, other variables can be timeStamp variables, which act like the time variable but have a different name.

- **Incompatibilities**
  - Constraints - Different types of data sources support different types of constraints. Most data sources don't support all of the types of constraints that tabledap advertises. For example, most data sources can't test `variable=~"RegularExpression"`. When a data source doesn't support a given type of constraint, tabledap gets extra data from the source, then does that constraint test itself. Sometimes, getting the extra data takes a lot of time or causes the remote server to throw an error.
  - File Types - Some results file types have restrictions. For example, .kml is only appropriate for results with longitude and latitude values. If a given request is incompatible with the requested file type, tabledap throws an error.

- **Requesting Compressed Files**

  ERDDAP doesn't offer results stored in compressed (e.g., .zip or .gzip) files. Instead, ERDDAP looks for accept-encoding in the HTTP GET request header sent by the client. If a supported compression type (gzip, x-gzip, or deflate) is found in the accept-encoding list, ERDDAP includes "content-encoding" in the HTTP response header and compresses the data as it transmits it. It is up to the client program to look for `content-encoding` and decompress the data accordingly. Requesting compression is optional, but compressed responses are often 3-10 times faster, so this is a big time savings if you are downloading lots of large files. (Note that there is no benefit to requesting compressed .png files since the files' contents are already compressed.)
  - By default, browsers and OPeNDAP clients always request compressed data and decompress the returned data.
  - With curl, add `--compressed` to the command line to tell curl to request a compressed response and automatically decompress it.
  - With other client software, you have explicitly set this up.
    Here is a Java example.
    Here is a Python example (although you should either handle `deflate`'d responses or not request `deflate`).

- **How to Cite a Dataset in a Paper**
  It is important to let readers of your paper know how you got the data that you used in your paper. For each dataset that you used, please look at the dataset's metadata in the Dataset Attribute Structure section at the bottom of the .html page for the dataset, e.g., https://coastwatch.pfeg.noaa.gov/erddap/griddap/jplMURSST41.html .
  The metadata sometimes includes a required or suggested citation format for the dataset. The "license" metadata sometimes lists restrictions on the use of the data.

  To generate a citation for a dataset:
  If you think of the dataset as a scientific article, you can generate a citation based on the author (see the "creator_name" or "institution" metadata), the date that you downloaded the data, the title (see the "title" metadata), and the publisher (see the "publisher_name" metadata). If possible, please include the specific URL(s) used to download the data. If the dataset's metadata includes a Digital Object Identifier (DOI), please include that in the citation you create.

- **Errors**
  Like other parts of the Internet (e.g., Tomcat, Apache, routers, your browser), ERDDAP uses various HTTP status/error codes to make it easy to distinguish different types of errors, including:
  - 400 Bad Request - for syntax errors in the request URL and other request errors that will never succeed. If you fix the stated problem, your request should succeed (or you'll get a different error message for a different problem).
  - 401 Unauthorized - when the user isn't currently authorized to access a given dataset. If you get this and think it unwarranted, please email the URL and error message to this ERDDAP's administrator to try to resolve the problem.
  - 403 Forbidden - when the user's IP address is on this ERDDAP's blacklist. If you get this and think it unwarranted, please email the URL and error message to this ERDDAP's administrator to try to resolve the problem.
  - 404 Not Found - for datasetID not found (perhaps temporarily), "Your query produced no matching results.", or other request errors. If the datasetID was not found but was there a few minutes ago, wait 15 minutes and try again.
  - 408 Timeout - for timeouts. Often, these can be fixed by making a request for less data, e.g., a shorter time period.
  - 413 Payload Too Large - when there isn't enough memory currently (or ever) available to process the response, or when the response is too large for the requested file type. Often, these can be fixed by making a request for less data, e.g., a shorter time period.
  - 416 Range Not Satisfiable - for invalid byte range requests. Note that ERDDAP's "files" system does not allow byte range requests to the individual .nc, .hdf, .bz2, .gz, .gzip, .tar, .tgz, .z, and .zip files (although the exact list may vary for different ERDDAP installations) because that approach is horribly inefficient (tens/hundreds/thousands of requests and the transfer of tons of data unnecessarily) and some of the client software is buggy in a way that causes problems for ERDDAP (tens/hundreds/thousands of open/broken sockets). Instead, either download the entire file (especially if you need repeated use of different parts of the file) or use the griddap or tabledap services to request a subset of a dataset via an (OPeN)DAP request. That's what DAP was designed for and does so well. Just because you can saw a log with a steak knife (sometimes) doesn't make it a good idea -- use your chainsaw.
  - 500 Internal Server Error - for errors that aren't the user's fault or responsibility. For code=500 errors that look like they should use a different code, or for errors that look like bugs in ERDDAP, please email the URL and error message to erd.data at noaa.gov .

  If the error response was generated by ERDDAP (not by some other part of the Internet, e.g., Tomcat, Apache, routers, or your browser), it will come with an (OPeN)DAPv2.0-formatted, plain text, UTF-8-encoded error message as the payload of the response, e.g.,
  ```
  Error {
      code=404;
      message="Not Found: Your query produced no matching results.
  (time>=2019-03-27T00:00:00Z is outside of the variable's actual_range:
  1970-02-26T20:00:00Z to 2019-03-26T15:00:00Z)";
  }
  ```
  Notes:

  - The `message` is a JSON-encoded string. The first few words, e.g., Not Found, are the generic name of the HTTP error code (e.g., HTTP error 404).
  - Of course, status code 200 OK (with the relevant payload) is used for completely successful requests.
  - Errors generated by other parts of the Internet (e.g., Tomcat, Apache, routers, or your browser) won't return an (OPeN)DAPv2.0-formatted, plain text, error message. In fact, they may not return any payload with the HTTP error code. In a browser, errors are obvious. But in a script, the best way to check for errors is to look at the HTTP status code first (is it 200=OK ?) and only secondarily see if there is an error message payload (which may not be (OPeN)DAPv2.0-formatted).
  - If you need help resolving a problem, email the URL and error message to this ERDDAP's administrator to try to resolve the problem. Or, you can join the ERDDAP Google Group / Mailing List and post the URL, the error message, and your question there.
  - If you see errors that have been assigned the wrong HTTP status code (especially code=500 errors that should be 4xx errors) or might be evidence of a bug in ERDDAP, please email the URL and error to erd.data at noaa.gov .

Contact Us

If you have questions, suggestions, or comments about ERDDAP in general (not this specific ERDDAP installation or its datasets), please send an email to `bob dot simons at noaa dot gov` and include the ERDDAP URL directly related to your question or comment.

Or, you can join the ERDDAP Google Group / Mailing List by visiting https://groups.google.com/forum/#!forum/erddap and clicking on "Apply for membership". Once you

are a member, you can post your question there or search to see if the question has already been asked and answered.

ERDDAP, Version 2.29.0
Disclaimers | Privacy Policy | Contact